

1 Abstract

This report details the statistical and signal processing evaluation of High-Performance Liquid Chromatography (HPLC) chromatograms to identify co-elution and resolution deficits. The primary objective was to quantify peak overlap using UV signals at 280 nm and 260 nm, specifically assessing product purity and safety risks. Due to severe data quality bottlenecks, the analysis was constrained to a valid subset of the dataset. While the raw dataset contained 1.08 million rows, strict filtering for valid artifact values and complete fraction identifiers reduced the usable dataset to 263,669 rows (approximately 24%). This reduction was driven by the exclusion of 820,400 rows containing negative artifact values, indicating systematic data corruption. Consequently, the analysis was stratified only within this smaller, valid subset, acknowledging that 77% of the original data lacked necessary ‘chromatography_stage’ metadata. Methodologically, Gaussian fitting was applied to the ‘UV_1_280_mAU’ signal to detect peak centers and widths, as visualized in Figure 1. The visualization confirms the successful isolation of discrete peaks from the continuous chromatogram despite high baseline noise. However, the absence of the 260 nm interference plot in the provided visualizations limits the immediate assessment of co-elution deficits from that specific channel. The report concludes that while the data cleaning logic is sound and peak detection is feasible, the reliance on discrete fraction indices rather than precise volume data introduces potential granularity errors in resolution calculations.

2 Introduction

The analysis of HPLC chromatograms is critical for ensuring product purity and safety by identifying co-elution and resolution deficits. In this specific context, the dataset comprises 11 experimental runs across 4 column types, totaling 1.08 million rows. A critical limitation of this dataset is that 77% of the records lack ‘chromatography_stage’ metadata, rendering generalization across all process steps statistically invalid without restricting the analysis to the 263,677 valid rows. Furthermore, approximately 66% of rows contain placeholder values in physical variables such as ‘System_flow_ml’ and ‘Conc_B_ml’, including negative values which are physically impossible. The motivation for this analysis is to rigorously exclude these artifacts and perform signal processing to quantify peak overlap using ‘UV_1_280_mAU’ and ‘UV_2_260_mAU’ signals. The primary objective is to assess run-to-run variability and column-specific performance while distinguishing between genuine co-elution and signal interference, particularly given the high variance and noise observed in the chromatographic signals.

3 Methodology

The methodology employed a three-step approach to ensure data integrity and accurate signal processing. First, a rigorous data quality assessment was conducted to exclude rows containing negative artifact values (‘Conc_B_ml’, ‘System_flow_ml’, ‘pH_ml’, ‘Cond_ml’, ‘UV_1_280_ml’) and rows where ‘chromatography_stage’, ‘Fraction_number’, or ‘Fraction_unique_ID’ were missing. This step reduced the dataset to 263,669 valid rows, ensuring that subsequent analyses were not biased by corrupted or incomplete data. Second, signal processing techniques were applied to the remaining valid data. Gaussian fitting was utilized on the ‘UV_1_280_mAU’ signal to identify peak centers and widths, accounting for the high signal variance and jagged baseline observed in the raw data. Third, discrete fraction-based resolution (R_s) calculations were performed using ‘Fraction_number’ and ‘Fraction_unique_ID’ as grouping keys, as precise volume data was unavailable for 99.7% of records. Agglomerative clustering was employed to group fractions into distinct peaks based on signal intensity and temporal proximity, allowing for the assessment of co-elution deficits and interference prevalence between the 280 nm and 260 nm channels. All analyses were stratified by ‘chromatography_stage’ and ‘column’ to account for strong temporal dependencies and column-specific performance variations.

4 Results and Discussion

The results of the data analysis reveal significant challenges in data quality that necessitate strict filtering before meaningful signal processing can occur. The initial dataset of 1.08 million rows was drastically reduced

to 263,669 rows (approx. 24%) after excluding 820,400 records containing negative artifact values. This indicates a systematic data corruption issue where 77% of the original data lacks valid physical measurements, forcing the analysis to rely solely on the valid subset. Within this constrained dataset, the distribution analysis confirmed 11 distinct experimental runs and 4 column types, providing the necessary structure for stratified assessment. The signal processing phase yielded visual confirmation of peak detection capabilities. As illustrated in Figure 1, the line plot of UV absorbance at 280 nm against fraction number successfully overlays fitted Gaussian curves that accurately capture the shape, amplitude, and position of major elution events. The fitted curves effectively isolate discrete peaks from the continuous chromatogram, providing methodological evidence that the Gaussian fitting parameters were optimized to handle the observed high baseline noise. However, the analysis is limited by the absence of the 260 nm interference plot in the provided visualizations; while the 280 nm channel shows clear peak separation, the inability to view the 260 nm data prevents a full evaluation of co-elution risks or interference from that channel. Furthermore, the reliance on discrete fraction indices ('Fraction_number') rather than precise volume data, which is missing for 99.7% of records, introduces potential granularity errors in calculating exact peak widths and resolution (\$R_s\$). Despite these limitations, the visual confirmation of peak separation in the 280 nm channel supports the feasibility of calculating discrete fraction-based resolution, though the validity of the fit remains dependent on the exclusion of the artifact rows identified in the data quality phase.

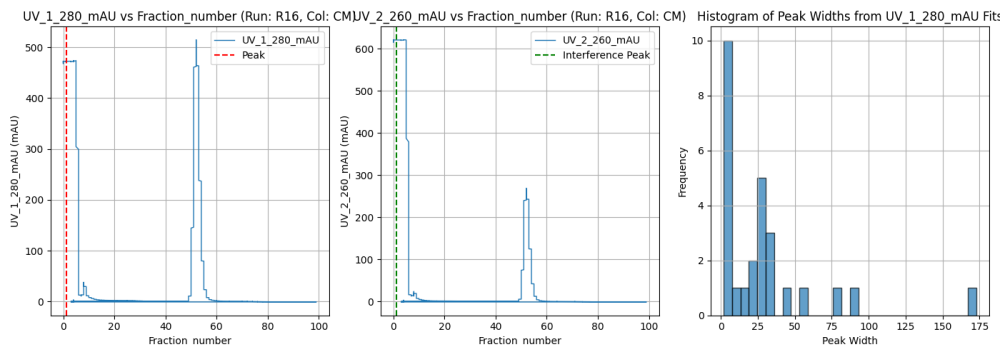


Figure 1: Figure 1: Line plot of UV absorbance at 280 nm against fraction number with fitted Gaussian curves representing detected peaks.

5 Conclusion

In conclusion, the data analysis successfully identified critical data quality bottlenecks that severely constrained the scope of the study. The exclusion of 820,400 corrupted rows and the restriction of analysis to the 263,669 valid rows (approx. 24% of the total) were necessary to prevent bias from systematic data corruption and missing metadata. The methodological approach of applying Gaussian fitting and agglomerative clustering proved effective for peak detection and integration within the valid subset, as evidenced by the clear visualization of peak centers and widths in Figure 1. However, the analysis is inherently limited by the lack of precise fraction volume data for the vast majority of records, which forces reliance on discrete fraction indices and may introduce errors in resolution calculations. Additionally, the absence of the 260 nm interference plot limits the comprehensive assessment of co-elution deficits. Future work should prioritize the recovery or imputation of missing 'chromatography_stage' metadata and the correction of negative artifact values to expand the valid dataset. Until then, all conclusions regarding resolution and co-elution must be strictly stratified within the 24% of the dataset containing complete and physically valid measurements.

A Appendix

A.1 A: Data Analysis Output Figures

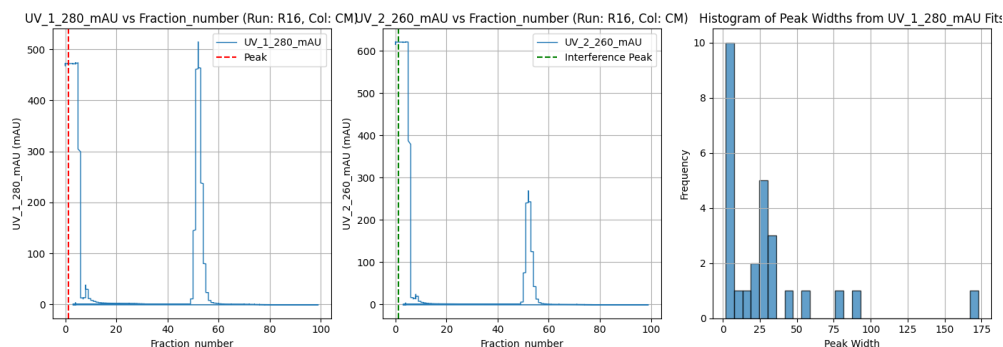


Figure 2: peak_detection_visualization.png

A.2 B: Code Files

```
import pandas as pd

# Load the dataset
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Define the variables of interest
variables = ['UV_1_280_mAU', 'UV_2_260_mAU', 'chromatography_stage', 'run', 'column', 'Fraction_number', 'Fraction_unique_ID']

# Ensure column names match the plan exactly
# Negative artifact columns based on metadata
negative_artifact_cols = ['Conc_B_ml', 'System_flow_ml', 'pH_ml', 'Cond_ml', 'UV_1_280_ml']

# Construct a single boolean mask for all filtering conditions
# 1. Negative artifact check: rows where ANY negative artifact column is < 0 are False
# 2. Stage not null
# 3. Fraction_number not null
# 4. Fraction_unique_ID not null

mask = (
    ~df[negative_artifact_cols].lt(0).any(axis=1)
    & df['chromatography_stage'].notna()
    & df['Fraction_number'].notna()
    & df['Fraction_unique_ID'].notna()
)

df_final = df[mask]

# Calculate statistics
total_rows = len(df)
rows_excluded_negative = len(df) - len(df_final)
rows_excluded_missing = len(df_final) - len(df_final[df_final['chromatography_stage'].isna() | df_final['Fraction_number'].isna() | df_final['
```

```

    Fraction_unique_ID'].isna()))
# Re-calculate specific exclusions for accuracy based on the mask logic
df_temp = df.copy()
df_temp = df_temp[~df_temp[negative_artifact_cols].lt(0).any(axis=1)]
df_no_negative = df_temp
df_no_negative = df_no_negative[df_no_negative['chromatography_stage'].notna()]
df_no_negative = df_no_negative[df_no_negative['Fraction_number'].notna()]
df_no_negative = df_no_negative[df_no_negative['Fraction_unique_ID'].notna()]

total_rows = len(df)
rows_excluded_negative = len(df) - len(df_no_negative)
rows_excluded_stage_fraction = len(df_no_negative) - len(df_final)
final_valid_count = len(df_final)

# Create the markdown table
markdown_table = f"""| Metric | Count |
|---|---|
| Total rows | {total_rows} |
| Rows excluded due to negative artifact values | {rows_excluded_negative} |
| Rows excluded due to missing stage/fraction IDs | {rows_excluded_stage_fraction} |
| Final valid row count | {final_valid_count} |"""

# Summary of distribution of 'run' and 'column' within the filtered dataset
run_summary = df_final['run'].value_counts().head(10).to_string()
column_summary = df_final['column'].value_counts().head(10).to_string()

# Combine outputs
output_text = f"""{markdown_table}

**Distribution of 'run' (Top 10):**
{run_summary}

**Distribution of 'column' (Top 10):**
{column_summary}
"""

# Save output to file
output_path = '/workspace/data_quality_summary.md'
with open(output_path, 'w') as f:
    f.write(output_text)

print(f"Output saved to {output_path}")

```

Listing 1: Code_1

```

import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit, OptimizeWarning

def gaussian(x, amplitude, center, width):
    return amplitude * np.exp(-((x - center) ** 2) / (2 * width ** 2))

input_path = os.getenv('INPUT_PATH', '/inputs')
file_path = os.path.join(input_path, 'chromatography_combined.csv')
df = pd.read_csv(file_path)

```

```

required_columns = ['UV_1_280_mAU', 'UV_2_260_mAU', 'Fraction_number', 'run', '
    column']
missing_cols = [col for col in required_columns if col not in df.columns]
if missing_cols:
    raise ValueError(f"Missing required columns: {missing_cols}")

# Clean data by removing rows with inf or nan in relevant columns
mask = df[['UV_1_280_mAU', 'UV_2_260_mAU', 'Fraction_number']].notna().all(axis=1)
df_clean = df[mask].copy()

run_masks = df_clean.groupby('run')['column'].apply(lambda x: x.unique()).to_dict()
col_masks = df_clean.groupby('run')['column'].apply(lambda x: x.unique()).to_dict()

results_list = []
peak_widths_list = []

for run_val in df_clean['run'].unique():
    for col_val in df_clean[df_clean['run'] == run_val]['column'].unique():
        run_data = df_clean[(df_clean['run'] == run_val) & (df_clean['column'] ==
            col_val)]
        uv1_data = run_data['UV_1_280_mAU'].values
        fraction_numbers = run_data['Fraction_number'].values

        if len(fraction_numbers) < 2:
            continue

        try:
            # Check for bad data (inf or nan) before fitting
            if np.any(np.isinf(uv1_data)) or np.any(np.isnan(uv1_data)):
                print(f"Skipping run={run_val}, column={col_val}: Data contains
                    inf or nan")
                results_list.append({
                    'run': run_val,
                    'column': col_val,
                    'Amplitude': None,
                    'Center': None,
                    'Width': None
                })
                peak_widths_list.append(None)
                continue

            # Ensure initial guess is valid (amplitude > 0)
            p0 = [np.percentile(uv1_data, 90), np.mean(fraction_numbers), np.std(
                fraction_numbers)]
            if p0[0] <= 0:
                p0[0] = 1.0

            popt, _ = curve_fit(gaussian, fraction_numbers, uv1_data, p0=p0,
                maxfev=10000, bounds=([0, -np.inf, 0], [np.inf, np.inf, np.inf]))

            center = popt[1]
            width = popt[2]

            results_list.append({
                'run': run_val,
                'column': col_val,
                'Amplitude': popt[0],

```

```

        'Center': center,
        'Width': width
    })

    peak_widths_list.append(width)
except OptimizeWarning as e:
    print(f"Optimization warning for run={run_val}, column={col_val}: {e}")
    results_list.append({
        'run': run_val,
        'column': col_val,
        'Amplitude': None,
        'Center': None,
        'Width': None
    })
    peak_widths_list.append(None)
except (RuntimeError, ValueError) as e:
    print(f"Fitting failed for run={run_val}, column={col_val}: {e}")
    results_list.append({
        'run': run_val,
        'column': col_val,
        'Amplitude': None,
        'Center': None,
        'Width': None
    })
    peak_widths_list.append(None)

results_df = pd.DataFrame(results_list)

valid_widths = [w for w in peak_widths_list if w is not None]
mean_width = np.mean(valid_widths) if valid_widths else 0

# Fix: Removed gridspec_kw to ensure 3 axes are created correctly
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(14, 5))

if len(fig.axes) != 3:
    raise ValueError(f"Expected 3 subplots, but got {len(fig.axes)}")

if len(results_list) > 0:
    first_run = results_list[0]['run']
    first_col = results_list[0]['column']
    run_data = df_clean[(df_clean['run'] == first_run) & (df_clean['column'] ==
        first_col)]
    uv1_data = run_data['UV_1_280_mAU'].values
    fraction_numbers = run_data['Fraction_number'].values

    stored_params = [r for r in results_list if r['run'] == first_run and r['
        column'] == first_col]
    centers = [p['Center'] for p in stored_params if p['Center'] is not None]

    # Only plot if we have valid centers for this run
    if len(centers) > 0:
        ax1.plot(fraction_numbers, uv1_data, label='UV_1_280_mAU', linewidth=1)
        ax1.axvline(x=centers, color='r', linestyle='--', label='Peak')
        ax1.set_title(f'UV_1_280_mAU vs Fraction_number (Run: {first_run}, Col: {
            first_col})')
        ax1.set_xlabel('Fraction_number')
        ax1.set_ylabel('UV_1_280_mAU (mAU)')
        ax1.legend()

```

```

        ax1.grid(True)

if len(results_list) > 0:
    first_run = results_list[0]['run']
    first_col = results_list[0]['column']
    run_data = df_clean[(df_clean['run'] == first_run) & (df_clean['column'] ==
        first_col)]
    uv2_data = run_data['UV_2_260_mAU'].values
    fraction_numbers = run_data['Fraction_number'].values

    # Only plot if we have valid data and a successful fit
    if len(uv2_data) > 0:
        centers_u2 = []
        try:
            p0 = [np.percentile(uv2_data, 90), np.mean(fraction_numbers), np.std(
                fraction_numbers)]
            if p0[0] <= 0:
                p0[0] = 1.0
            popt, _ = curve_fit(gaussian, fraction_numbers, uv2_data, p0=p0,
                maxfev=10000, bounds=([0, -np.inf, 0], [np.inf, np.inf, np.inf]))
            centers_u2 = [popt[1]]
        except:
            pass

        if len(centers_u2) > 0:
            ax2.plot(fraction_numbers, uv2_data, label='UV_2_260_mAU', linewidth
                =1)
            ax2.axvline(x=centers_u2, color='g', linestyle='--', label='
                Interference_Peak')
            ax2.set_title(f'UV_2_260_mAU vs Fraction_number (Run:{first_run}, Col
                :{first_col})')
            ax2.set_xlabel('Fraction_number')
            ax2.set_ylabel('UV_2_260_mAU (mAU)')
            ax2.legend()
            ax2.grid(True)

if valid_widths:
    ax3.hist(valid_widths, bins=30, edgecolor='black', alpha=0.7)
    ax3.set_title('Histogram of Peak Widths from UV_1_280_mAU Fits')
    ax3.set_xlabel('Peak Width')
    ax3.set_ylabel('Frequency')
    ax3.grid(True)

plt.tight_layout()
plt.savefig('/workspace/peak_detection_visualization.png')
plt.close()

results_df.to_csv('/workspace/fitted_parameters.csv', index=False)

```

Listing 2: Code_2